ENGINE BENCHMARKING SYSTEM

LEGEND :
- (green) : CORE SYSTEMS I BUILD
- (blue) : COMPUTE / DERIVED FROM SYSTEMS
- (red) : ASSUMED
- RED TEXT : DESCRIBES DATA SHAPES

I. SYNTHETIC INPUT GENERATION

$[D, V_D], [A_c, V_{Ac}], [A_f], [P], [P_A] \longrightarrow [(D \times V_D) + (A_c \times V_{Ac}) + A_f, (P + P_A)]$

TOTAL INPUT VARIATIONS ; TOTAL PERSONAS

D: QUERY DIMENSIONS
V_D: VARIATIONS ON EACH D ,

A_c: CUSTOM ADVERSARIAL DIMS
V_{Ac}: VARIATIONS ON EACH A_c ,
A_f : FIXED REUSABLE ADVERSARIAL INPUTS

FOR MULTI-TURN
P: TARGET USER PERSONAS
P_A : ADVERSARIAL USER PERSONAS

II. INPUT DATASET → INPUTS HARNESS → TARGET AI APP → OUTPUTS DATASET] → TRACES

(THIN WRAPPER FOR BATCH INPUTS TO TARGET APP)

$[N] \longrightarrow [N, M], [N, M, T]$

INPUTS ; OUTPUTS ; TRACES

N = PERSONAS × VARIATIONS

N : INPUTS
M : TURNS

T: TRACE DIMENSIONS

III. TRACES → ABLATION ENGINE → MODIFIED TRACES , GROUND TRUTH ISSUEBOARD

$[N, M, T] \longrightarrow [N, M, T^*], [N, E_k]$

TRACES

T^*: ABLATED TRACES WITH KNOWN ERRORS ;

E_k: A LIST OF KNOWN ERRORS THAT EACH INPUT WAS ABLATED WITH. (1 : SINGLE ERROR ; >1 : COMPOUND ERRORS)

EACH ERROR HAS {TITLE, DESC, CATEGORY, SEVERITY, ERROR_ID}

ISSUEBOARD IS {TRACE_ID, ERROR_ID} VIEW OF OUTPUT

IV. MODIFIED TRACES → ENGINE → OUTPUT: PREDICTED ISSUEBOARD

$[N, M, T^*] \longrightarrow [N, E_p]$; E_p : ENGINE PREDICTS ERRORS ON TRACES.

V. {GROUND TRUTH ISSUEBOARD, PREDICTED ISSUEBOARD} → SCORERS → ENGINE PERFORMANCE BENCHMARK REPORT

$[N, E_k], [N, E_p] \longrightarrow$ BENCHMARK NUMBERS

I. SYNTHETIC DATA GENERATION . ON THE TARGET APP (PRE-PROD)

$[D, V_D], [A_c, V_{AC}], [A_F] \longrightarrow [(D \times V_D) + (A \times V_{AC}) + A_F]$

INPUT VARIATIONS

OPTIONAL

OR $[P], [P_A], [D, V_D], [A, V_{AC}], [A_F] \longrightarrow [(P + P_A), (D \times V_D) + (A \times V_{AC}) + A_F]$

PERSONAS SCENARIOS

IA. SINGLE TURN.

INPUTS : $[D, V_D$: D ORTHOGONAL INPUT QUERY DIMENSIONS, EACH WITH V_D VARIATIONS$]$

OUTPUT : $D \times V_D$ TOTAL SAFE QUERY INPUTS

IB. MULTI - TURN

INPUTS : $[P$: P PERSONA DESCRIPTIONS$]$

I_C . ADVERSARIAL

INPUTS : $[A_c, V_{AC}$: A_c CUSTOM APP SPECIFIC ADVERSARIAL VARIATION DIMENSIONS, EACH W V_{AC} VARIATIONS$]$

OUTPUTS : $(A_c \times V_{AC}) + A_F$: A_F FIXED REUSABLE ADVERSARIAL INPUTS

* HOW CAN THIS PROCESS BE TRUSTED?

- IDEALLY THE SYNTHETIC INPUT DATA DISTRIBUTION GENERATED (D_s) SHOULD BE REPRESENTATIVE OF REAL WORLD DATA DISTRIBUTION (D_o)

$$D_o : P(x) = \int P(x|\theta_o) P(\theta_o) d\theta \; ; \; \theta_o = \{P_o, I_o\} \; ; \; I_o = \{D_o, A_o\}$$

REAL-WORLD PROMPTS OBSERVED 'x' ARE BASED ON HOW REAL-WORLD PERSONAS INTERACT WITH THE APP θ_o, I_o AND WHAT REAL-WORLD PERSONAS ARE ENCOUNTERED.

- THE GOAL OF SYNTHETIC DATA GENERATOR IS TO ESTIMATE P, I SUCH THAT THEY ARE EQUIVALENT OR SUPERSETS OF P_o, I_o

THE ACTUAL TEST OF WHETHER SYNTHETIC DATA GENERATOR ACCOMPLISHED THIS GOAL CAN ONLY COME AFTER HAVING REAL-WORLD OBSERVATIONS ON INPUT VARIATIONS. AT THAT POINT WE CAN DO 3 COMPARISONS TO BENCHMARK OUR SYNTHETIC GENERATOR :

1. COMPARE $P(\theta)$ VS. $P(\theta_o)$ i.e. WERE OUR PRIORS WELL-ESTIMATED? (SIMILAR PERSONAS, TYPES OF Qs)
2. COMPARE $P(x)$ VS. $P(x_o)$ i.e. ARE THE INPUT PROMPTS SIMILAR TO ESTIMATE? (LENGTH, SEMANTIC, etc.)
3. COMPARE $P(x|\theta)$ VS. $P(x_o|\theta_o)$ i.e. IN A SUB-CATEGORY OF A PRIOR, DO THE OBS. INPUT PROMPTS FOLLOW THE ESTIMATED DISTRIBUTION?

II. GENERATING OUTPUTS & TRACES.

$[N] \longrightarrow [N, M], [N, M, T]$

II.A. INPUTS DO NOT HAVE PERSONAS (i.e. SINGLE TURN)

- SIMPLE BATCHING HARNESS, RUNS BATCHED INPUTS AGAINST TARGET APP'S ENDPOINT GENERATING OUTPUTS & TRACES.

$N = (D \times V_D) + (A_C \times V_{AC}) + A_F$

$M = 1$

$[N] \longrightarrow [N], [N, T]$

II.B. INPUTS HAVE PERSONAS (i.e. MULTI-TURN)

- LOADS AN LLM WITH GIVEN PERSONA AND VARIATION AND INTERACTS FOR MAX_TURNS WITH THE TARGET AI APP'S ENDPOINT GENERATING MULTI-TURN CONVERSATIONS & CORRESPONDING TRACES.

$D_1 = (D \times V_D)$

$D_2 = (A_C \times V_{AC}) + A_F$

$N = (P \times D_1) + (P_A \times D_2)$; PERSONA-WISE VARIATION SPLIT

$M = [1, \text{MAX\_TURNS}]$

$[N] \longrightarrow [N, M], [N, M, T]$

III. THE ABLATION ENGINE

[N, M, T] ⟶ [N, M, T*], [N, E]

GOAL: INJECT KNOWN ERRORS FROM A SET E INTO TRACES. GENERATE GROUND TRUTH BY STORING THE INJECTIONS.

NUANCE: THE INPUT TRACES MIGHT HAVE UNKNOWN ERRORS ALREADY PRESENT IN THE TRACES. ABLATIONS SHOULD BE COMPATIBLE WITH THOSE.

III A. BASE-CASE : ASSUMES TRACES ARE GOLDEN i.e. NO HIDDEN UNKNOWN ERRORS.

STEP 1 : GENERATE CUSTOM ERRORS FROM KNOWN HIGH LEVEL ERROR CATEGORIES CE & TRACES

IN : [N, M, T], CE ⟶ OUT : [E, CE]

ERROR : { ERROR_ID : UNIQUE IDENTIFIER FOR THIS ERROR,
TITLE : SINGLE PHRASE TITLE DESCRIBING THIS ERROR,
DESC : VERBOSE DESCRIPTION OF THIS ERROR,
CATEGORY : PARENT CATEGORY TO WHICH THIS ERROR BELONGS TO,
SEVERITY : 3-CLASS CATEGORY THIS ERROR IS CLASSIFIED AS [LOW, MED, HIGH] }

USES AGENT WITH SDK TOOLS TO EXPLORE TRACES

STEP 2 : GENERATE A TRACE FILTER AND ABLATION STRATEGY

IN : [N, M, T], [E, CE] ⟶ OUT : [E, CE]

ERROR : { ERROR_ID : UNIQUE ID,
FILTER : SELECTION STRATEGY ON TRACE OBJECT TO IDENTIFY ELIGIBLE TRACES IN WHICH ERROR CAN EXIST. IS A LIST OF SDK ACTIONS ON TRACE PROPERTIES,
ABLATION_ACTIONS : A LIST OF STR IN, STR OUT FUNCTIONS TO APPLY ON FILTERED TRACES TO ABLATE THEM. }

STEP 3 : TEST EACH ERROR'S FILTER AND ABLATION STRATEGY FOR ERRORS.

FILTER SHOULD RETURN ATLEAST 5 ELIGIBLE TRACES
ABLATION_ACTION SHOULD RUN WITHOUT ERROR VIA A PRIMITIVE
SURFACE ALL ERRORS & RETURN TO STEP 2.

STEP 4 : CREATE ABLATIONS ON ORIGINAL TRACES AND CREATE ISSUEBOARD GROUND TRUTH

IN : [N, M, T], E ⟶ OUT : [N, M, T*], [N, E]

FOR EACH ERROR E :

- APPLY FILTER ON TRACES
- SUB-SAMPLE IF TOO LARGE
- APPLY ABLATIONS
- STORE {TRACE_ID, ERROR} INFO FOR GROUND TRUTH

III B. WHAT CHANGES IF TRACES HAVE HIDDEN UNKNOWN ISSUES.

- THESE HIDDEN ISSUES ARE UNDISCOVERED IN THE IB GROUND TRUTH

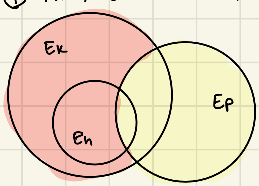
  IDEALLY: FOR TRACES $[N, M, T]$, ALREADY HAVE ERRORS $[N, E_h]$

  WITH ABLATION WE INTRODUCE KNOWN NEW ERRORS:

  $[N, M, T^*] \longrightarrow [N, E_h + E_k]$

- THE PROCESS OF ABLATION WILL:

  1. FULLY OVERWRITE E_h: IN WHICH CASE NO CONCERN AS E_k IS GROUND TRUTH TO EVALUATE E_p

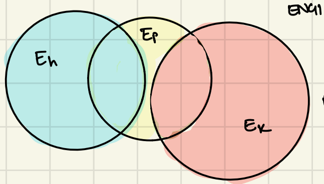


$E_k \cap E_p$: TRUE POSITIVE = ACTUAL TP.

$E_p - E_k$: FALSE POSITIVE = ACTUAL FP

$E_k - E_p$: FALSE NEGATIVE = ACTUAL FN

  2. NOT AT ALL OVERWRITE E_h: IN WHICH CASE THE GROUND TRUTH CREATES IS BIASED EVALUATOR OF ENGINE. BECAUSE



$E_k \cap E_p$ = PRED TRUE POS. (UNDER-ESTIMATED)

$E_k \cap E_p + E_h \cap E_p$ = ACTUAL TRUE POS.

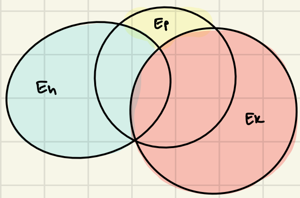
$E_p - E_k$ = EST. FALSE POSITIVE (OVER-EST.)

$E_p - E_k - E_h$ = ACTUAL FP.

$E_k - E_p$ = EST. FALSE NEGATIVE (UNDER EST.)

$E_k + E_h - E_p$ = ACTUAL FN.

  3. PARTIALLY OVERWRITE E_h: SAME ANALYSIS AS 2 BUT WITH POTENTIALLY LESSER DEVIATIONS FROM TRUE EVALUATIONS.



* CONCLUSION

- ABLATION STRATEGY IS A BIASED ESTIMATOR OF ENGINE'S PERFORMANCE. THIS IS EXPECTED.
- THE ONLY FOOL-PROOF WAY TO FIND E_h IS EXPERT ANNOTATIONS. ABLATIONS CAN BE CONTINUALLY IMPROVED AS MORE ANNOTATIONS ARE REVEALED.
- THE BIAS IS A DIRECT FUNCTION OF HOW MANY E_h ARE MISSED BY ABLATION. THIS CAN BE MINIMIZED BY INCREASING TEST-TIME COMPUTE & HYPER PARAMS ON ABLATIONS.

IV. ENGINE

- I BUILD A SIMPLE PROMPT BASED DEEP_AGENT THAT HAS TOOL CALL ACCESS AND INSTRUCTIONS ON HOW TO REPORT ERRORS.
- RUNS SEQUENTIALLY ON EVERY TRACE AND MAINTAINS RUNNING LIST OF ERRORS.
- AT THE END — A META DEEP_AGENT TAKES ANOTHER PASS TO CONSOLIDATE ERRORS FOR FINAL OUTPUT.

V. SCORERS

① ERROR-CATEGORY TRACE CLASSIFICATION : PER-CATEGORY PRECISION, RECALL, F1, COHEN'S KAPPA

ERROR-MATCHING ALGO :

WITHIN A CATEGORY, BASED ON TITLE & DESCRIPTIONS, CREATE A $E_p \rightarrow E_k$ MAP.

- TEXT-BASED MATCH MATCHING (COSINE SIM., TF-IDF VECTOR MATCH)

ONCE PREDICTED ERRORS ARE MAPPED TO KNOWN ERRORS

② SUB-CATEGORY PER-ERROR TRACE CLASSIFICATION : PRECISION, RECALL, F1, COHEN'S-K

③ CROSS-ERROR SEVERITY CLASSIFICATION : ASYMMETRIC LOSS THAT PUNISHES HIGHER ACTUAL SEVERITY PREDICTED AS LOWER MORE (EX-SQUARED) VS. ACTUAL LOW PREDICTED AS HIGH (SCORED LINEAR)

④ DESCRIPTION DEVIATION : CAN BE A SCORE OR FAILURE MODES ID WITH PASS/FAIL